Lista de Exercícios 2 - Análise de Algoritmos e Caracterização de Tempos de Execução - Algoritmos Iterativos e Dividir e Conquistar

Disciplina: PCC104 - Projeto de Análise de Algoritmos

A lista de exercícios a seguir está baseada no livro-texto adotado na disciplina, disponível na biblioteca digital da UFOP. Para acessá-lo, entre no sistema MinhaUFOP e navegue até: Biblioteca Digital → E-books Minha Biblioteca. Em seguida, localize a obra utilizando a referência indicada abaixo:

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Algoritmos: teoria e prática. 4. ed. Rio de Janeiro: Grupo GEN | LTC, 2022.

1. Considere o algoritmo abaixo, que calcula a soma dos elementos de um arranjo $A[1\,..\,n]$.

```
SOMA-ARRANJO(A, n)
s ← 0
for i ← 1 to n do
    s ← s + A[i]
end for
return s
```

   (a) Defina o tamanho da entrada do algoritmo.

   (b) Construa a tabela de custos no estilo CLRS, associando a cada linha um custo c_i e o número de vezes que ela é executada em função de n.

   (c) A partir da tabela, escreva a expressão de $T(n)$ como uma soma dos termos c_i ponderados por suas frequências.

   (d) Simplifique $T(n)$, escrevendo-a na forma $T(n) = an + b$, e identifique as constantes a e b em termos dos custos c_i.

   (e) Justifique formalmente que $T(n) = \Theta(n)$, exibindo constantes $c_1, c_2 > 0$ e $n_0 \geq 1$ que satisfaçam a definição de Θ.

   (f) Este algoritmo possui melhor caso e pior caso distintos? Justifique.

2. Considere o algoritmo abaixo, que verifica se o arranjo $A[1\,..\,n]$ contém o valor x, retornando o índice da primeira ocorrência ou -1 caso x não esteja em A.

```
BUSCA-LINEAR(A, n, x)
1: i ← 1
2: while i ≤ n and A[i] ≠ x do
3:     i ← i + 1
4: end while
5: if i ≤ n then
6:     return i
7: else
8:     return −1
9: end if
```

   (a) Defina o tamanho da entrada do algoritmo.

   (b) Monte a tabela de custos no estilo CLRS. Use t para denotar o número de vezes que o teste do laço **while** (linha 2) é avaliado, e expresse o número de execuções das linhas 2 e 3 em função de t.

   (c) Escreva $T(n)$ em função dos custos c_i e de t.

   (d) **Melhor caso:** qual é o valor de t? Descreva uma instância que produza esse caso e mostre que $T_{\text{melhor}}(n) = \Theta(1)$.

   (e) **Pior caso:** qual é o valor de t? Descreva todas as instâncias (em função de A e x) que produzem esse caso e mostre que $T_{\text{pior}}(n) = \Theta(n)$.

   (f) **Caso médio:** suponha que x ocorra exatamente uma vez em A e que a probabilidade de x estar em qualquer posição $i \in \{1, 2, \ldots, n\}$ seja uniforme. Calcule $\mathbb{E}[t]$ e mostre que $T_{\text{med}}(n) = \Theta(n)$.

3. Considere o algoritmo abaixo, que verifica se o arranjo $A[1\,..\,n]$ contém algum par de elementos iguais.

```
TEM-DUPLICATA(A, n)
for i ← 1 to n − 1 do
    for j ← i + 1 to n do
        if A[i] = A[j] then
            return VERDADEIRO
        end if
    end for
end for
return FALSO
```

(a) Defina o tamanho da entrada do algoritmo.

(b) Determine o número de vezes que o teste do laço interno (linha 2) é executado no **pior caso**, em função de n. Descreva uma instância que produza o pior caso.

Sugestão: para cada valor de i, conte quantos valores de j são considerados e some sobre i. Use a identidade $\sum_{i=1}^{n-1}(n-i) = \dfrac{n(n-1)}{2}$.

(c) Construa a tabela de custos no estilo CLRS para o pior caso e escreva $T_{\text{pior}}(n)$ como soma dos termos c_i ponderados por suas frequências.

(d) Mostre que $T_{\text{pior}}(n) = \Theta(n^2)$, exibindo as constantes da definição assintótica.

(e) Descreva o **melhor caso** do algoritmo: quantas execuções da linha 3 ocorrem? Qual é a classe assintótica de $T_{\text{melhor}}(n)$?

(f) Compare a classe assintótica deste algoritmo com a do Insertion-Sort no pior caso.

4. Resolva as seguintes relações de recorrência:

(a) $x(n) = x(n-1) + 5$ para $n > 1$, $x(1) = 0$

(b) $x(n) = 3x(n-1)$ para $n > 1$, $x(1) = 4$

(c) $x(n) = x(n-1) + n$ para $n > 0$, $x(0) = 0$

(d) $x(n) = x(n/2) + n$ para $n > 1$, $x(1) = 1$ (resolver para $n = 2^k$)

(e) $x(n) = x(n/3) + 1$ para $n > 1$, $x(1) = 1$ (resolver para $n = 3^k$)

5. Projete um algoritmo para computar 2^n para qualquer inteiro não negativo, n, baseado na fórmula $2^n = 2^{n-1} + 2^{n-1}$.

(a) Apresente a relação de recorrência para o número de adições feitas pelo algoritmo e resolva a relação.

(b) Desenhe a árvore de chamadas recursivas para este algoritmo e conte o número de chamadas feita pelo algoritmo.

(c) Este é um bom algoritmo para resolver este problema.

6. Utilize o método de substituição para determinar um limite assintótico superior para (Ver Seção 4.3 do livro texto):

$$T(n) = 2T(\lfloor n/2 \rfloor) + \Theta(n)$$

7. Utilize o método de árvore de recursão para descobrir um limite superior assintótico para a recorrência (Ver Seção 4.4 do livro texto):

$$T(n) = T(n/3) + T(2n/3) + n$$

8. Considere o algoritmo recursivo abaixo, que calcula o fatorial de um inteiro $n \geq 0$.

```
FATORIAL(n)
1: if n = 0 then
2:     return 1
3: end if
4: return n · FATORIAL(n − 1)
```

(a) Defina o tamanho da entrada do algoritmo.

(b) Seja $T(n)$ o tempo de execução de Fatorial(n). Escreva a relação de recorrência para $T(n)$, justificando o custo de cada linha e indicando o caso base.

(c) Resolva a recorrência pelo método da iteração (substituição reversa), obtendo uma expressão fechada para $T(n)$.

(d) Classifique $T(n)$ usando a notação Θ.

(e) Desenhe a árvore de chamadas recursivas para $n = 5$ e indique a profundidade da árvore em função de n.

9. Considere o algoritmo recursivo abaixo, que realiza busca binária em um arranjo $A[p\,..\,r]$ ordenado em ordem não decrescente, retornando o índice de x ou -1 caso x não esteja em A.

```
BUSCA-BINÁRIA(A, p, r, x)
if p > r then
    return −1
end if
q ← ⌊(p + r)/2⌋
if A[q] = x then
    return q
else if A[q] > x then
    return BUSCA-BINÁRIA(A, p, q − 1, x)
else
    return BUSCA-BINÁRIA(A, q + 1, r, x)
end if
```

Considere $n = r - p + 1$ o tamanho do subarranjo na chamada corrente.

(a) Justifique por que cada chamada recursiva opera sobre um subarranjo de tamanho no máximo $\lfloor n/2 \rfloor$.

(b) Escreva a relação de recorrência $T(n)$ para o **pior caso**, indicando claramente o caso base e o custo (não recursivo) de cada chamada.

(c) Resolva a recorrência pelo método da iteração para $n = 2^k$, obtendo uma expressão fechada para $T(n)$.

(d) Mostre que $T(n) = \Theta(\log n)$.

(e) Descreva o **melhor caso** do algoritmo e dê sua classe assintótica.

(f) Aplique o Teorema Mestre para confirmar o resultado obtido pelo método da iteração. Identifique a, b e $f(n)$, e indique qual caso do teorema se aplica.

10. Considere o algoritmo recursivo abaixo, que retorna o valor máximo de um arranjo $A[p\,..\,r]$ pelo método de divisão e conquista.

```
MÁXIMO(A, p, r)
if p = r then
    return A[p]
end if
q ← ⌊(p + r)/2⌋
m1 ← MÁXIMO(A, p, q)
m2 ← MÁXIMO(A, q + 1, r)
if m1 ≥ m2 then
    return m1
else
    return m2
end if
```

Considere $n = r - p + 1$ o tamanho do subarranjo na chamada corrente.

(a) Identifique os passos de **dividir**, **conquistar** e **combinar** no algoritmo, no espírito do paradigma de divisão e conquista.

(b) Escreva a relação de recorrência $T(n)$, justificando o número de chamadas recursivas, o tamanho de cada subproblema e o custo de combinar os resultados.

(c) Resolva a recorrência pelo método da árvore de recursão para $n = 2^k$. Indique o número de níveis, o custo por nível e o custo total.

(d) Mostre que $T(n) = \Theta(n)$.

(e) Aplique o Teorema Mestre para confirmar o resultado obtido pela árvore de recursão. Identifique a, b e $f(n)$, e indique qual caso do teorema se aplica.

(f) Compare a classe assintótica deste algoritmo com a de uma versão iterativa que percorre o arranjo uma única vez. O que isto sugere sobre o ganho (ou ausência de ganho) ao usar divisão e conquista neste problema?

11. **Implementação do Heapsort.** Implemente o algoritmo HEAPSORT em uma linguagem de programação de sua preferência (C, C++, Python, Java ou Rust), seguindo **exatamente** a estrutura modular do pseudocódigo apresentado em aula **Oservação: No dia da prova você deverá entregar o código impresso juntamente com a prova. Algumas das questões farão referência ao seu próprio código. A não entrega do código implica em nota 0 nestas questões.**:

```
MAX-HEAPIFY(A, i)
ℓ ← LEFT(i)
r ← RIGHT(i)
if ℓ ≤ A.heap-size e A[ℓ] > A[i] then
    largest ← ℓ
else
    largest ← i
end if
if r ≤ A.heap-size e A[r] > A[largest] then
    largest ← r
end if
if largest ≠ i then
    troca A[i] ↔ A[largest]
    MAX-HEAPIFY(A, largest)
end if
```

```
BUILD-MAX-HEAP(A)
A.heap-size ← A.length
for i ← ⌊A.length/2⌋ até 1 do
    MAX-HEAPIFY(A, i)
end for
```

```
HEAPSORT(A)
BUILD-MAX-HEAP(A)
for i ← A.length até 2 do
    troca A[1] ↔ A[i]
    A.heap-size ← A.heap-size − 1
    MAX-HEAPIFY(A, 1)
end for
```

Requisitos da implementação:

(a) Implemente as três rotinas MAX-HEAPIFY, BUILD-MAX-HEAP e HEAPSORT como funções/métodos separados, preservando os nomes e as assinaturas.

(b) Mantenha `heap-size` como um atributo separado de `length` (use uma struct, classe ou variáveis globais, conforme a linguagem). Não os trate como sinônimos.

(c) A ordenação deve ser feita **in place** — nenhum arranjo auxiliar de tamanho proporcional a n é permitido.

(d) **Atenção à indexação:** o pseudocódigo do CLRS usa indexação a partir de 1. Se sua linguagem indexa a partir de 0 (C, Python, etc.), documente claramente no início do código a convenção adotada e seja consistente em todas as rotinas.

(e) Inclua um programa principal que: (i) lê um arranjo de entrada, (ii) executa HEAPSORT, (iii) imprime o arranjo ordenado.

(f) **Testes:** valide a implementação com pelo menos quatro casos — arranjo vazio, arranjo unitário, arranjo já ordenado, arranjo em ordem decrescente — e um caso aleatório com $n \geq 20$ elementos.

(g) Entregue o código-fonte e um arquivo `README` contendo: instruções de compilação/execução e um exemplo de saída para cada caso de teste.

12. **Implementação do Quicksort.** Implemente o algoritmo QUICKSORT (com particionamento de Lomuto), na mesma linguagem escolhida para o exercício anterior, seguindo o pseudocódigo apresentado em aula **Oservação: No dia da prova você deverá entregar o código impresso juntamente com a prova. Algumas das questões farão referência ao seu próprio código. A não entrega do código implica em nota 0 nestas questões.**:

```
PARTITION(A, p, r)
x ← A[r]
i ← p − 1
for j ← p até r − 1 do
    if A[j] ≤ x then
        i ← i + 1
        troca A[i] ↔ A[j]
    end if
end for
troca A[i + 1] ↔ A[r]
return i + 1
```

```
QUICKSORT(A, p, r)
if p < r then
    q ← PARTITION(A, p, r)
    QUICKSORT(A, p, q − 1)
    QUICKSORT(A, q + 1, r)
end if
```

Requisitos da implementação:

(a) Implemente PARTITION e QUICKSORT como funções/métodos separados, preservando os nomes e as assinaturas.

(b) A ordenação deve ser **in place** — não aloque arranjos auxiliares para os subproblemas.

(c) Inclua um programa principal que recebe o arranjo, chama QUICKSORT$(A, 1, n)$ (ou a versão equivalente em indexação 0) e imprime o arranjo ordenado.

(d) **Testes:** valide com os mesmos casos do exercício anterior (vazio, unitário, ordenado, decrescente, aleatório com $n \geq 20$). Observe e relate em uma seção do `README` o que acontece com a entrada **já ordenada** em termos de tempo de execução.

(e) **Item bônus:** implemente também RANDOMIZED-PARTITION (que escolhe o pivô uniformemente em $\{p, p+1, \ldots, r\}$ e o troca com $A[r]$ antes de chamar PARTITION) e a variante RANDOMIZED-QUICKSORT. Repita o teste do item anterior na entrada já ordenada e compare empiricamente o tempo de execução com o da versão determinística.

13. Considere o algoritmo QUICKSORT utilizando o último elemento como pivô:

```
QUICKSORT(A, p, r)
if p < r then
    q ← PARTITION(A, p, r)
    QUICKSORT(A, p, q − 1)
    QUICKSORT(A, q + 1, r)
end if
PARTITION(A, p, r)
x ← A[r]
i ← p − 1
for j ← p até r − 1 do
    if A[j] ≤ x then
        i ← i + 1
        troca A[i] ↔ A[j]
    end if
end for
troca A[i + 1] ↔ A[r]
return i + 1
```

Em todos os itens abaixo, considere $n = r - p + 1$ o tamanho do subarranjo na chamada corrente.

(a) **Custo de PARTITION.** Construa a tabela de custos no estilo CLRS para PARTITION, atribuindo um custo c_i a cada linha e contando o número de execuções em função de n. Mostre que PARTITION executa em tempo $\Theta(n)$.

(b) **Pior caso.** Suponha que, em **toda** chamada recursiva, o particionamento produza um subarranjo vazio e outro de tamanho $n - 1$.

i. Descreva uma instância de tamanho n que produza esse comportamento (descreva também o conteúdo de A que dispara o pior caso). *Dica: pense no que acontece quando $A[r]$ é sempre escolhido como pivô.*

ii. Escreva a relação de recorrência $T(n)$ para o pior caso, indicando o caso base.

iii. Resolva a recorrência pelo método da iteração e mostre que $T(n) = \Theta(n^2)$.

iv. Verifique que o Teorema Mestre **não** se aplica a esta recorrência. Justifique por que.

(c) **Melhor caso (particionamento balanceado).** Suponha agora que, em toda chamada recursiva, o particionamento produza dois subarranjos de tamanho aproximadamente $n/2$.

i. Escreva a relação de recorrência $T(n)$ para esse caso.

ii. Resolva pelo método da árvore de recursão para $n = 2^k$. Indique o número de níveis, o custo por nível e o custo total.

iii. Aplique o Teorema Mestre e identifique a, b e $f(n)$. Indique qual caso do teorema se aplica e conclua que $T(n) = \Theta(n \lg n)$.

(d) **Particionamento desbalanceado mas em proporção constante.** Suponha que toda chamada produza um subarranjo de tamanho αn e outro de tamanho $(1 - \alpha)n$, com $\alpha \in (0, 1/2]$ constante.

i. Escreva a recorrência $T(n)$.

ii. Esboce a árvore de recursão e argumente que o custo por nível é $O(n)$.

iii. Mostre que a profundidade da árvore (medida pelo ramo mais longo, de tamanho $(1-\alpha)^k n$) é $\Theta(\lg n)$.

iv. Conclua que $T(n) = O(n \lg n)$.

v. *Discussão:* qual a implicação prática deste resultado? Por que é importante que o algoritmo seja eficiente mesmo com particionamentos consistentemente desbalanceados (desde que em proporção constante)?